

CS-4347 Airport Management System —

Milestone 3

A desktop database application for querying and managing airport flight data, built with Python and MySQL. The GUI provides five features: flight lookup, trip planning, aircraft utilization reporting, seat availability checking, and seat booking.

Tech-Stack

Language: Python 3.10 or newer **Database:** MySQL Server 8.0 or newer **Third-party dependency:** mysql-connector-python >= 8.0

Install dependencies

```
python3 -m pip install -r requirements.txt
```

Configure the database connection

Open config.ini and fill in your MySQL credentials:

```
[database]
host      = localhost
port      = 3306
user      = root
password  = your_password_here
database  = airport_db
```

If password is left blank, the application will prompt for it at startup.

Create the schema and load data (first time only)

```
python3 setup_database.py
```

This script drops and recreates the `airport_db` database, applies `schema.sql`, bulk-loads all CSV files from `data/`, and inserts a small set of demo reservations so the booking and itinerary features have sample data to work with.

Launch the application

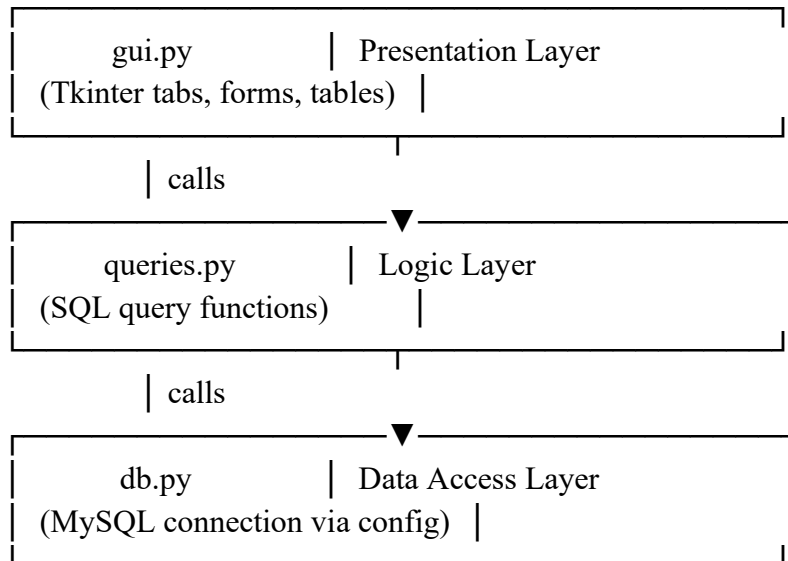
```
python3 gui.py
```

A green dot labeled **MySQL** in the top-right corner of the window confirms a successful database connection. A command-line interface is also available via `app.py` for all Milestone 2 query features.

Project structure

Architecture

The project follows a three-layer architecture that separates the GUI, SQL query functions, and database connection handling into different files.



Each layer has a single responsibility. `gui.py` handles all user interaction and never writes SQL. `queries.py` contains every SQL statement and returns plain Python dictionaries. `db.py` manages the connection lifecycle and reads credentials from `config.ini`. This means any layer can be changed independently. For example, swapping the GUI framework or switching database engines would only touch one file each.

Database queries run in background threads via a shared `_run_async` helper in `BaseTab`, so the GUI stays responsive while loading results. Results are posted back to the main thread using Tkinter's `after()` callback, which is the correct pattern for thread-safe UI updates.

Language Choice — Python

Python was chosen because it works well with MySQL and made it easier to build the backend logic and GUI using Tkinter. The `mysql-connector-python` library provides a simple way to connect python with the MySQL database and executes queries safely using parameterized statements. Also, python can run on multiple operating systems without a compilation step, which simplifies setup for grading and demonstration.

GUI Framework — Tkinter

Tkinter is Python's built-in GUI library, meaning no additional installation is required beyond the database driver. The `ttk.Notebook` widget made it straightforward to organize the application into tabs for each feature. Tkinter also worked well for building forms, tables, dropdown menus, and

other interface elements needed for the database system. Since this project mainly focuses on data entry and query results, Tkinter was a practical choice that kept the application simple and easy to run.

Menu and Layout Design

The application is organized into separate tabs, with each tab representing one major feature of the system such as flight lookup, trip search, aircraft utilization, seat booking, and passenger itinerary lookup. This design was chosen because each feature is a fully self-contained workflow. A user searching for a trip wouldn't need to interact with the aircraft utilization report, and mixing them would create clutter. Separating the features into tabs keeps the interface more organized and prevents too much information from appearing on a single screen. It also makes the application easier to navigate during testing and demonstrations.

Within each tab, **labeled text entry fields** are used for free-form inputs like flight numbers, dates, and passenger names, because these values cannot be enumerated in advance. **Dropdown menus (Comboboxes)** are used where the valid choices are known at runtime, especially for selecting flight legs and available seats in the booking form. The options are loaded dynamically from the database after a seat check. This prevents the user from selecting an invalid seat number or a leg that does not exist. **Buttons** are used to trigger actions and are labeled with verbs (*Search Flight*, *Check Seats*, *Book Seat*) to make the expected behavior clear.

A shared status bar at the bottom of the window provides feedback on every action without using disruptive popup dialogs for routine results.

Schema Design

The schema follows the textbook airport ER model and augments it in two places to support the booking feature.

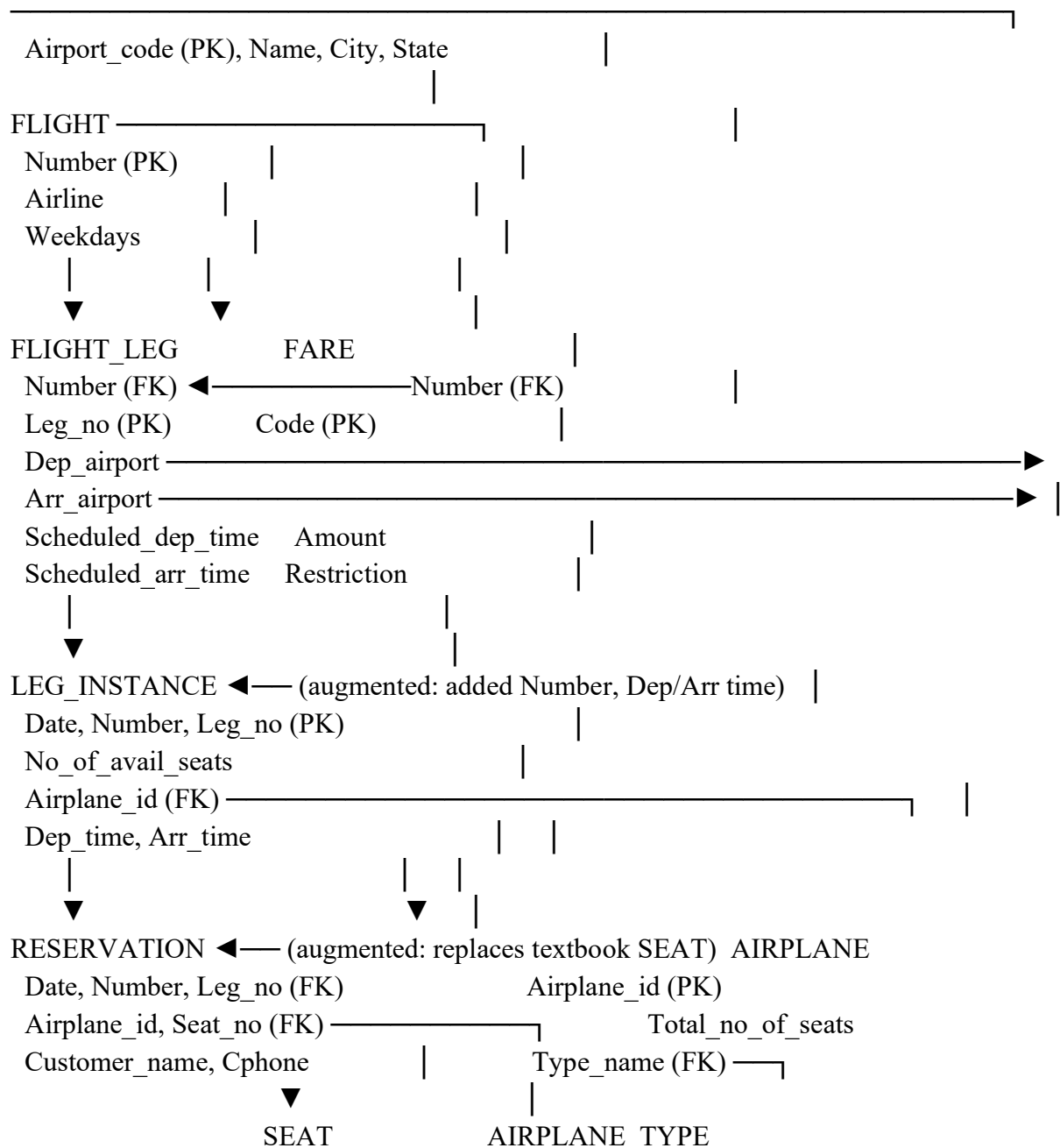
The textbook SEAT table (which combined seat identity with passenger assignment) was split into two tables:

- **SEAT** — stores the physical seats on an airplane (Airplane_id, Seat_no, Class). This is static data that reflects the aircraft's configuration.
- **RESERVATION** — stores individual bookings (Date, Number, Leg_no, Airplane_id, Seat_no, Customer_name, Cphone). This is the transactional table that grows as seats are sold.

This split makes seat availability queries straightforward: available seats are simply SEAT rows for the assigned airplane that have no matching RESERVATION row for that flight date and leg. It also allows the system to enforce uniqueness at the database level via a primary key on (Date, Number, Leg_no, Seat_no), which prevents double-booking even under concurrent requests.

The LEG_INSTANCE table was extended with Number (flight number), Dep_time, and Arr_time to link instances back to their flight and record actual departure and arrival times separately from the scheduled times in FLIGHT_LEG.

AIRPORT



Airplane_id (FK)	Type_name (PK)
Seat_no (PK)	Company
Class	Max_seats

Quick Start Guide

1. Flight Lookup

Go to the **Flight** tab. Enter a flight number (e.g. 1000) and a date (e.g. 2025-10-04), then click **Search Flight**. The results show the flight header at the top, a leg table with departure and arrival airports and times, and a fare table below.

2. Trip Search

Go to the **Trip** tab. Enter an origin and destination using either airport codes (DFW, MEX) or city names (Dallas, Mexico City), and enter a date. Click **Search Trip**. The top table lists all direct flights on that date; the bottom table lists one-stop connecting flights with at least one hour of layover time at the connecting airport.

3. Aircraft Utilization Report

Go to the **Utilization** tab. Enter a start date and end date, then click **Run Report**. The table lists every airplane by registration number and type along with the total number of flight leg instances it operated during that period.

4. Seat Availability and Booking

Go to the **Seats** tab. Enter a flight number and date, then click **Check Seats**. The summary cards at the top show total capacity, confirmed bookings, and remaining seats across all legs. The detail table breaks this down per leg.

To book a seat, use the **BOOK A SEAT** section below the table:

1. Select a leg from the **Leg** dropdown.
2. Select an available seat from the **Available Seat** dropdown (only unbooked seats appear).
3. Enter the passenger name and phone number.
4. Click **Book Seat**.

A confirmation message appears below the button and the availability counts refresh automatically.

5. Passenger Itinerary

Go to the **Itinerary** tab. Enter a passenger name (e.g. John Smith) or phone number, then click **Find Itinerary**. The table shows every flight leg the passenger is booked on, including departure and arrival airports, scheduled times, and seat assignment.